ChatGPT

Comprehensive Capabilities and Architecture for a macOS LLM Assistant

Personal Productivity Functions

- **Task & Reminder Management:** Create and manage to-dos or reminders via the macOS Reminders app. *Example prompt:* "**Remind me to call John at 3 PM today**." – *Script:* Use AppleScript (Reminders) to make a new reminder with the given title and time.
- **Calendar Events & Scheduling:** Add or query calendar events using the Calendar app. *Example prompt:* "**Schedule a meeting tomorrow at 10 AM titled 'Project Sync'**" or "**What's on my calendar for today?**" – *Script:* AppleScript to create an event or fetch today's events (or Python with Calendar API for online calendars).
- **Email Composition & Queries:** Draft and send emails, or check inbox for specific senders, through Apple Mail. *Example prompt:* "**Email Alice that I'll be 10 minutes late**" or "**Do I have any new emails from Bob?**" – *Script:* AppleScript controlling **Mail** to compose/send a message (or use Python's SMTP/IMAP for internet email). AppleScript can automate email tasks, as it can interact with apps to send emails and more [1].
- **Note Taking:** Create or search notes in the Notes app for quick memos. *Example prompt:* "**Take a note titled 'Ideas' and write 'Research LLM architecture improvements'**." – *Script:* AppleScript (Notes) to make a new note with specified title and content.
- **Contacts & Communication:** Query contact info or initiate messages. *Example prompt:* "**Find Alice's phone number**" – *Script:* AppleScript (Contacts) to retrieve contact details. Or send an iMessage via AppleScript (Messages app) e.g. "**Text Bob that I'm on my way**."
- **Focus & Productivity Aids:** Automate focus modes or timers. *Example prompt:* "**Turn on Do Not Disturb for 1 hour**" – *Script:* Shell or AppleScript via **System Events** to enable Focus mode. Similarly, could start a Pomodoro timer via a Python script or Automator workflow.

Development and Coding Tasks

- **Build and Compile Projects:** Run build commands or compile code bases. *Example prompt:* "**Build my Xcode project**" or "**Run the Maven build for the Java project**." – *Script:* Utilize shell commands (via `osascript` or Python `subprocess`) to run `xcodebuild`, `make`, `npm run build`, etc., possibly in the appropriate directory. This can be executed by telling Terminal to run the command (AppleScript's `do shell script` can invoke build tools [2]).
- **Run Tests and Analysis:** Execute test suites or linters. *Example prompt:* "**Run unit tests for the project**" – *Script:* Shell commands (e.g., `pytest`, `npm test`, `xcodebuild test`) initiated via AppleScript or direct shell execution. The assistant can capture output and report results.
- **Git Version Control:** Automate Git operations through command-line. *Example prompts:* "**What's the status of my repo?**", "**Commit and push all changes with message 'Fix bug'**", or "**Switch to the develop branch**." – *Script:* Shell (Git CLI) to run `git status`, `git add/commit/push`, `git checkout` etc., and then relay the output. This speeds up development workflows by voice.

- **Launching Dev Servers/Services:** Start or stop local development servers and environments. *Example prompt:* "**Launch the local server**" or "**Stop the Docker containers**." – *Script:* AppleScript to open Terminal and run commands like `npm start`, `python manage.py runserver`, `docker compose up`, etc., or directly via a shell script. The assistant could maintain a library of common dev commands to execute on request.
- **Code Editing & Navigation:** Open files in editors or perform search/replace. *Example prompt:* "**Open the file `app.py` in VSCode**" or "**Find 'TODO' comments in the project**." – *Script:* Use the `open` command (shell) to launch files in apps (e.g., `open -a "Visual Studio Code" app.py`), or use AppleScript if the editor is scriptable. For search, run shell commands like `grep` or utilize macOS Spotlight search programmatically to locate files.
- **Development Environment Management:** Other tasks such as setting environment variables, clearing caches, or opening documentation. *Example prompt:* "**Set NODE_ENV to production**" – *Script:* A shell command to export an env variable (applied to relevant processes). Or "**Open the Django documentation**" – *Script:* AppleScript to open a web browser to the docs.

System Automation and OS Control

- **File Management:** Perform file operations (open, move, copy, rename, delete). *Example prompts:* "**Open the Downloads folder**", "**Move all PDFs from Downloads to Documents**", or "**Rename file `report.doc` to `2025 Report.doc`**." – *Script:* AppleScript via **Finder** to manipulate files (e.g., use Finder's `move` command) or shell commands (`mv`, `cp`, etc.). For example, an AppleScript can move files between folders in an automated loop [3].
- **Application Launching & Quitting:** Open or close applications on command. *Example prompts:* "**Launch Safari**", "**Quit Microsoft Word**". – *Script:* AppleScript `tell application "AppName" to activate` (or to `quit`) [4]. This allows voice-activated app switching. You can also bring specific apps to front or close all apps (by iterating through running applications in AppleScript).
- **Window Management:** Basic control of windows and UI. *Example prompts:* "**Minimize all windows**", "**Snap this window to the left half**", "**Switch to the next desktop space**." – *Script:* Use **System Events** in AppleScript to manipulate GUIs (e.g., iterate over windows to minimize or position them). Complex window tiling might require third-party tools (which could be invoked via shell).
- **System Settings Adjustment:** Change system preferences and hardware settings. *Example prompts:* "**Increase the volume to 50%**", "**Dim the display**", "**Toggle Wi-Fi**", or "**Turn on Dark Mode**." – *Script:* AppleScript can change volume or mute audio (`set volume` command) [5], toggle dark mode via System Events [6], or even toggle Wi-Fi and other settings [7]. For example, one script can set Dark Mode to on/off [6], and another can toggle Wi-Fi via System Preferences scripting [7]. This gives full voice control over system toggles.
- **OS Commands and Utilities:** Trigger various OS-level actions. *Examples:* "**Empty the Trash**" – uses AppleScript (`tell application "Finder" to empty trash` [8]); "**Take a screenshot**" – uses a shell utility (`screencapture` command) and perhaps provides the file path; "**Shut down the computer**" or "**Restart now**" – uses AppleScript through **System Events** to shut down or reboot without confirmation [9]. Other examples: lock the screen, log out, or put system to sleep via AppleScript (`tell app "System Events" to sleep`).
- **Clipboard and Text Utilities:** Manage clipboard content or dictate text. *Example:* "**What's on my clipboard?**" – the assistant can use AppleScript to get the clipboard text and read it out. Or "**Copy this text to clipboard**" – put a given string into clipboard via AppleScript. It can also use macOS Text-to-Speech (say command) to read text aloud if needed, or conversely convert speech to text (though that's handled by Whisper in this architecture).

Basic Third-Party App Control

- **Spotify Music Playback:** Control music apps like Spotify through scripts. *Example prompts:* "**Play some Beatles music on Spotify**", "**Pause the music**", "**Skip this track**", or "**Set Spotify volume to 70%**." – *Script:* AppleScript can directly command Spotify (e.g., `tell application "Spotify" to playpause` or `next track` [10] [11]). This covers play/pause, track navigation, volume adjustment, or playing specific playlists (by name or URL).
- **Slack Messaging:** Send quick messages or status updates on Slack. *Example prompt:* "**Send a Slack message to #team that I'm running late**." – *Script:* Since Slack isn't natively AppleScript-friendly, use Slack's API via a Python script or shell `curl` command. The assistant would format a JSON payload to Slack's webhook or use a Slack Python client with an API token to post the message. (For reading messages or notifications, one could integrate Slack's API as well, or use macOS notifications if Slack posts those.)
- **Web Browsers & Online Content:** Control Chrome/Safari for specific tasks. *Example prompts:* "**Open YouTube and play lo-fi music**", "**Search Google for 'macOS automation tips'**" – *Script:* AppleScript can tell **Safari** or **Chrome** to open a URL or perform a search query. For example, `tell application "Safari" to open location "<URL>"`. This effectively automates third-party browsers. The assistant could also fill out forms or click links via JavaScript injection or GUI scripting, though those are advanced use cases.
- **Communication & Meetings:** Basic control of communication apps. *Example:* "**Join my next Zoom meeting**" – *Script:* If a Zoom link is known, AppleScript or shell can open it (which launches Zoom). Or use GUI scripting to click the "Join" button if the app is open. For Microsoft Teams or others, similar approaches (or keyboard shortcuts via AppleScript System Events to mute/unmute, etc.).
- **Other Apps Integration:** The assistant can integrate with many apps given the right script or API:
- *Task Managers (Third-party):* If using Things, Todoist, etc., it could create tasks via their URL schemes or APIs.
- *Notes Apps:* For Evernote/Notion, it could use their APIs or AppleScript if available (Evernote had AppleScript support).
- *Media and Creative Apps:* e.g., control playback in VLC, start/stop recordings, or automate Photoshop with AppleScript or JavaScript for Automation.
- *Home Automation:* If HomeKit devices are present, use the `homekit` command or an API to toggle lights, although Home app itself might not be scriptable, a Python script could use HomeKit frameworks or send requests to smart home hubs.

Internet-Based Actions and Services

- **Web Search Queries:** Perform online searches and return results. *Example prompt:* "**Search the web for the latest AI news**" or "**Google 'best Italian restaurants in NYC'.**" – *Script:* The assistant might open the default browser with the search query (as a quick solution), or use a web search API to fetch top results and summarize them. This utilizes internet connectivity to answer general questions or fetch information not stored locally.
- **Weather Information:** Retrieve current weather or forecasts for a location. *Example prompt:* "**What's the weather tomorrow in London?**" – *Script:* Use a weather API (e.g., OpenWeatherMap) via a Python script to get forecast data, then have the assistant speak or display "Tomorrow's forecast...". This requires internet but gives up-to-date info.
- **Online Email and Calendar Services:** If using Gmail/Outlook or online calendars: check email, send messages, or manage Google Calendar. *Example prompt:* "**Check my Gmail for new messages from**

HR" or "**Add an event to my Google Calendar on Monday at 2 PM**." – *Script:* Python with Gmail API or IMAP to fetch emails (filtering by sender), and Google Calendar API to insert events. (If using local Mail/Calendar apps, the assistant could do it via AppleScript as noted in Personal Productivity, but here it's directly via web services for broader support.)

- **News and Knowledge Lookup:** Get current news headlines or factual answers. *Example prompt:* "**What's the latest headline from The New York Times?**" or "**Who won the game last night?**" – *Script:* A Python script could call a news API or scrape a news site's RSS feed to retrieve headlines. For general knowledge Q&A, the assistant might use an online knowledge base or Wikipedia API: e.g., fetch a summary from Wikipedia for a factual question.
- **Online Utilities:** Various other internet-based tasks:
- *Translation:* "**Translate 'good morning' to French**" – call a translation API (Google Translate or similar) and return the result.
- *Currency/Unit Conversion:* "**Convert 100 USD to EUR**" – call an exchange rate API.
- *Stock Quotes:* "**What's Apple's stock price?**" – use a finance API to get real-time quotes.
- *Maps and Directions:* "**How do I get to 123 Main St by car?**" – integrate with Google Maps API or Apple Maps (via URL schemes) to get directions or travel time.
- *Web Automation:* "**Fill out my timesheet online**" – this could involve a script using something like Selenium (though that's heavyweight) or AppleScript controlling a browser if the site is predictable.

(Each of the above functionalities can be backed by an appropriate script: AppleScript is ideal for controlling local apps and macOS UI, shell/Python scripts are best for web APIs, file operations, and CLI tools. In many cases, a combination is used – e.g., AppleScript to orchestrate an app and Python to handle data processing or API calls.)

Architecture Considerations and Recommendations

- **LLM Model Choice – Size vs Performance:** The current use of **Mistral 7B** via llama.cpp is a strong choice for an entirely local setup. Mistral 7B is widely regarded as one of the most powerful open 7B models – it even *outperforms* older 13B models like Llama-2 13B on many benchmarks [12] [13], thanks to optimizations like grouped-query attention that boost efficiency [14]. This means you get surprisingly good language and coding abilities from a smaller model that fits in memory. However, being only 7B parameters, it will still be less accurate and knowledgeable compared to larger models (for instance, it's noted to be significantly behind GPT-3.5/4 in general capability [12]). If offline operation is paramount, Mistral 7B (or its fine-tuned instruct variants) is optimal for speed. If you find accuracy lacking and are open to larger local models, a 13B or 20B model (such as Llama-2 13B, CodeLlama 13B, or newer open models) could improve understanding and reliability at the cost of slower inference. On Apple Silicon with llama.cpp, 13B models roughly double memory and may run ~half the speed of 7B. Consider hardware limits: on 16GB RAM Macs, 13B in 4-bit quantization is feasible but near the limit. For maximum accuracy (and if cloud access is acceptable), you could leverage an API to GPT-4 or similar for the NLP part – though that sacrifices the fully local advantage. In summary, the Mistral 7B choice is a sweet spot for local use [15], and moving to a larger model is only advisable if you need better comprehension and can tolerate slower responses or have a more powerful Mac. Quantizing the model (e.g. 4-bit or 5-bit) and using Metal acceleration in llama.cpp are recommended to maximize inference speed on Mac [16].

- **Whisper for Speech-to-Text:** Using **whisper.cpp** is a sensible approach for voice input. Whisper (especially tiny or base models) can run in real-time on Apple Silicon, providing decent accuracy. If you need faster transcription with a slight hit to accuracy, you can use a smaller Whisper model or

even Apple's built-in Dictation (though that may be less flexible). For better accuracy in understanding commands, the `base.en` or `small.en` Whisper model balances speed and correctness. Since the user experience depends on quick responses, ensure you load the Whisper model into memory at startup to avoid delays, and possibly segment audio for streaming transcription if needed. Whisper is state-of-the-art for offline STT, so the architecture here is solid. There's no need for a drastic change, but keep it in mind that for multi-language support or faster-than-real-time needs, you might adjust the model size or explore alternatives like Mozilla DeepSpeech for specific domains (though they likely won't beat Whisper's quality).

- **Structured JSON Output & Prompt Engineering:** The assistant's design of producing structured JSON instructions is excellent for modularity — it cleanly separates *what* the model decides to do from *how* it's executed. To improve reliability in JSON generation, consider refining the prompt or system message given to Mistral. For example, you can provide few-shot examples of user commands and the correct JSON outputs, or explicitly instruct the model (in a system prompt) to **only** produce a JSON with a specific schema and no extra commentary. OpenAI's function calling approach has shown that with proper guidance, models can return well-formed JSON consistently. While Mistral doesn't natively have "function calling", you can emulate this by training or prompting it to output JSON. If you notice the model sometimes gives free-form text or malformed JSON, you might:

  - Include a brief JSON schema or examples in the prompt (e.g., "When you respond, output a JSON like `{"action": "...", "parameters": {...}}` and nothing else.").
  - Use temperature settings cautiously (a lower temperature like 0.2–0.3 to reduce creativity so it sticks to format).
  - Post-process the output: e.g., validate JSON and if invalid, automatically re-prompt the model with something like "Format your answer as JSON only." This fallback can enhance reliability.

Additionally, as you add more capabilities, the JSON schema might need to expand (more action types or parameters). Ensure the prompt dynamically reflects the available actions. You might maintain a list in the system prompt: e.g., *"You can output actions:* `open_app`, `send_email`, `run_shell`, *etc., with these fields..."* so the model is aware of new abilities without full retraining. This prompt-engineering approach allows **ease of expansion** – you just update the model's instructions as you integrate new scripts.

- **Extensibility via Modular Scripts (Plugin-Like Design):** To maximize ease of expansion, architect the system so that adding a new skill is as simple as writing a new script and updating a registry. For example, maintain a library or directory of scriptable actions (AppleScript files, shell/Python scripts, Automator workflows) each identified by an action name. The JSON output from the model can include an `"action": "name_of_action"` and parameters; your Python backend then maps that to the corresponding script to execute. This is analogous to a plugin system or how **LangChain**/**Agent** frameworks handle tools. It means the LLM doesn't need the exact code for every task, just a high-level command. For instance, you add a "`control_spotify`" action that takes parameters like `"command": "next_track"` – once the model knows this action exists (from prompt or fine-tuning), it can invoke it, and your code runs the AppleScript for Spotify. This separation keeps the core logic clean and makes it easy to add or remove capabilities. It also improves maintainability: each script can be tested independently, and the model's job is just to choose the right action. As your list of actions grows, consider grouping or namespacing them (e.g., prefix with app or domain: `dev.git_commit`, `sys.set_volume` etc.) to avoid confusion. If the list becomes very large, you

might implement a two-step approach: first use the LLM to classify the request or select a tool, then maybe fill in details – but a well-crafted single prompt can often handle it by outputting the correct action name and args in one go.

- **Model Fine-Tuning and Specialization:** If you find the base model isn't sufficiently accurate in understanding your commands or producing the desired JSON, you could **fine-tune Mistral 7B** on a custom dataset of instructions. For example, gather a training set where the "user" prompt is a voice command and the "assistant" output is the JSON action. Fine-tuning (perhaps via LoRA adapters to keep it lightweight) can significantly improve the model's reliability in the narrow domain of "macOS assistant commands". Recent work shows that fine-tuning Mistral 7B on specific data (even on Apple Silicon hardware) is feasible [17]. This would essentially teach the model the format and the kind of responses more explicitly, resulting in higher accuracy. You can use tools like Hugging Face's Transformers + PEFT or Axolotl for this. Keep in mind fine-tuning on a Mac, while possible, can be slow and might require an M2 Pro/Max with lots of RAM or an M3. If fine-tuning is not practical, another approach is **few-shot prompting**: provide a few example Q&A pairs in the prompt so the model sees what to do. In tandem, ensure you're using an instruction-tuned model variant. If you haven't already, you might try an *instruct fine-tuned* version of Mistral (e.g., `mistral-7b-instruct`), which is optimized for conversational and instructional tasks [18]. Such models are better at following user instructions and could boost accuracy out-of-the-box.

- **Knowledge Expansion and Retrieval:** One limitation of any standalone LLM (especially a 7B one) is the fixed knowledge cutoff. Your assistant might not know specific app commands or information that isn't in its training data (e.g., details of macOS Sonoma if not trained on it, or your personal data). To mitigate this without a larger model, consider **Retrieval-Augmented Generation (RAG)**. This means integrating a vector database or search step: for example, if the user asks a question that requires documentation (say, "Clean my Downloads folder of files older than 30 days"), the system could first search a knowledge base (maybe AppleScript dictionaries or StackOverflow) for relevant snippets, then feed that into the model's context. In practice, you could index Apple's developer docs or common AppleScript recipes and let the assistant pull in relevant info when needed. This can greatly expand what the 7B model can handle, without having all knowledge in the model's weights. It also aids extensibility: when you add new features (like integration with a new app), you can store a description of that app's commands and some examples in an external memory. The model can retrieve those and thereby "learn" new capabilities on the fly, rather than relying purely on its internal training. RAG, however, adds complexity – you'd need to implement the retrieval step and ensure the prompt includes the retrieved text. It's an enhancement to consider if your assistant needs to answer more open-ended queries or work with lots of dynamic data.

- **Alternate Architectural Approaches:** The current architecture is fairly straightforward: Speech -> Text (Whisper), Text -> JSON (LLM), JSON -> Action execution. This is effective and mirrors designs used in some local voice assistant projects. Alternatives or additions you might consider:

- **Agent-based frameworks:** Instead of a single JSON-outputting model, you could use an agent loop where the LLM can iterate, reasoning about complex tasks. For example, using a library like LangChain or LlamaIndex, the assistant could break down multi-step requests. If a user asks, "**Find all PDF files in my Downloads and email them to me**," an agent approach might have the LLM generate a plan: (1) search for PDFs, (2) compress them, (3) compose an email). It could then execute each in turn. In your current design, you'd likely handle this by pre-defining a script that does the

entire sequence, or by calling the model multiple times. An agent framework can give the model more autonomy to handle novel multi-step tasks. The trade-off is increased complexity and slower runtime, so this is only worthwhile if you anticipate very complex queries.

- **Use of High-Level APIs or Shortcuts:** Apple's **Shortcuts** app (Automator's successor) allows chaining actions in a user-friendly way. An architecture might involve the LLM choosing or constructing a Shortcut to run. However, programmatically creating shortcuts is non-trivial, so sticking to scripts is usually simpler.

- **Cloud Augmentation:** Even if the core remains Mistral 7B, you could selectively use cloud services for heavy tasks. For example, for a very complex natural language query or code generation task, the system could route to an OpenAI API. Or use cloud STT if Whisper struggles with a particular accent/language. A "hybrid" architecture can be more flexible (e.g., local by default, cloud for fallback), though again at the cost of complexity and requiring internet.

- **Inference Optimizations:** To improve latency of the LLM responses, ensure you leverage Apple Silicon optimizations. llama.cpp has Metal support – compile it with `METAL=1` to use the GPU, which can significantly boost token generation speed on M1/M2. Running the model at 4-bit quantization (and maybe using the grouped-query attention variant of the model) can yield >20 tokens/second on an M2, which is decent [16]. If that's still not fast enough for real-time interaction, consider response shortening techniques: have the model output just the JSON (no extra text) which it already does, and perhaps limit the length of responses by design. You can also experiment with **Core ML** conversion (Apple has tools to convert models to Core ML format to run on the Neural Engine) – though support for Mistral 7B specifically might be bleeding edge, Core ML could run inference even faster on the ANE. Keep an eye on projects like **MLC (Machine Learning Compilation)** which aim to optimize LLM runtime on various hardware. In short, you likely can get the voice assistant responding in a second or two for short commands with the right optimizations, which is key for a good experience.

- **Accuracy and Error Handling:** Even with a good model and prompts, mistakes will happen (e.g., misheard STT or the LLM misinterpreting a command). Architect for robustness: include confirmation steps for destructive actions ("Did you mean to delete all files in Downloads?" could be a follow-up query from the assistant). You could implement a confidence check – for example, if Whisper's confidence is low or if the LLM's output score is low, have the assistant ask for clarification. Logging and testing are crucial: log each command, the parsed JSON, and the outcome, so you can refine the system over time where it consistently misunderstands. Over time, you might discover certain phrasing that confuses the model; you can address that by adding more examples to the prompt or training data. Leverage the JSON structure to your advantage: since the model outputs a structured action, you can systematically validate it (e.g., check that required fields exist and types are correct), providing an opportunity to catch errors before execution. If an invalid action is produced, you can fall back to a safe response or ask the user to rephrase.

Comparison Summary: The **Mistral 7B + whisper.cpp + JSON** approach is a cutting-edge setup for a fully local macOS "Jarvis." It offers privacy and offline reliability, and with Mistral's strong performance-per-size, it's quite capable [15]. The primary limitations are in *accuracy* (where larger or fine-tuned models or cloud APIs would win) and *knowledge breadth* (where retrieval or a larger model might help). The architecture can be improved by fine-tuning the model for instruction-following, optimizing inference (Metal and quantization), and designing the system to be easily extensible with new scripts (a plugin-like modular

design). Also consider incorporating retrieval mechanisms for up-to-date information or documentation, and possibly an agent-like approach for multi-step tasks if user requests grow in complexity. By iterating on prompts and possibly training, you can steadily increase the assistant's **accuracy** and **capabilities** without sacrificing the convenient JSON-based command execution structure. Overall, the chosen stack is close to optimal for the goals; the recommendations above aim to refine it – making the assistant faster, smarter, and easier to expand as you add more macOS controls and integrations. [12] [1]

---

[1] [3] [4] [5] [6] [7] [8] [9] Boosting Productivity and Automation With AppleScript on macOS
https://safjan.com/Boosting%20Productivity%20and%20Automation%20with%20AppleScript%20on%20macOS/

[2] Automate tasks using AppleScript and Terminal on Mac – Apple Support (AU)
https://support.apple.com/guide/terminal/automate-tasks-using-applescript-and-terminal-trml1003/mac

[10] [11] AppleScript for Spotify - dhruv's wiki
https://wiki.dhruvs.space/tools/mac/applescript/applescript-for-spotify/

[12] Running Mistral 7B LLM with llama.cpp on Apple Silicon | acordier
https://acordier.org/posts/mistral-7b-llama-cpp/

[13] [14] [18] Mistral 7B: Best Open Source LLM So Far
http://anakin.ai/blog/mistral-7b/

[15] [17] Fine-Tuning Mistral-7B on Apple Silicon: A Mac User's Journey with Axolotl & LoRA | by Plawan Rath | May, 2025 | Medium
https://medium.com/@plawanrath/fine-tuning-mistral-7b-on-apple-silicon-a-mac-users-journey-with-axolotl-lora-c6ff53858e7d

[16] llama.cpp with mistral-7b-instruct-v0.2.Q5_K_M.gguf ... - GitHub
https://github.com/ggerganov/llama.cpp/issues/5619